

<

HCMC UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY



HOMEWORK REPORT

COURSE: SOFTWARE TESTING

Assignment: GUI & Usability Testing trên EMS (Event Management System)

STUDENT INFORMATION

Field	Detailed Information
Full Name	Cao Trần Bá Đạt
Student ID	23127168
Class Section	23KTPM1

Selected APIs for Testing

No.	Feature ID & Name	Pool	Target Endpoint	HTTP Method
1	FR-06: Product Detail View	Pool A	/api/products/:id	GET
2	FR-11: Order History View (User)	Pool B	/api/orders/my-orders	GET
3	FR-16: Product Import from CSV	Pool C	/api/admin/import-products	POST

Pool A:

- API endpoint: `GET /api/products/:id`
- Feature: FR-06 - Product Detail (Xem chi tiết một sản phẩm)
- Audit: All test cases are VALID, except for TC-03 (INCOMPLETE), TC-04 (INCOMPLETE), TC-05 (INCOMPLETE), TC-05 to TC-30 (INCOMPLETE), TC-33 (INCOMPLETE), TC-34 (INCOMPLETE), TC-35 (INVALID), and TC-36 (INCOMPLETE).
 - TC-03, TC-04, and TC-05 are INCOMPLETE because the AI assumed arbitrary database seed sizes ($N = 50$, $N - 1 = 49$, $N + 1 = 51$), which must be mapped to the actual database state where maximum existing product ID is $N = 5$.
 - TC-05 to TC-30, TC-34, and TC-35 are INCOMPLETE because the AI hardcoded rigid error payload assertions (`{"error": "<string>"}`), which are not strictly specified in the API contract and cannot be asserted deterministically.
 - TC-33 is INCOMPLETE because it lacked the concrete Pre-request Script needed to authenticate as a regular user (`test@eshop.com`) and attach the Bearer token for HTTP verb tampering.
 - TC-34 is INCOMPLETE because it only tested `id=1`, which was already covered in TC-01; testing with another valid product entity (`id=2`) provides better schema verification.
 - TC-35 is INVALID because the AI generated a NoSQL/MongoDB injection payload (`{"$ne": null}`), which is irrelevant to the SQLite/SQL backend of this system.
 - TC-36 is INCOMPLETE because the AI framed it only as a redundant general schema check on `id=1`, missing the explicit data type and boundary assertions (`typeof price === 'number'` and `price > 0`) on other products like `id=2`.
- Correction:
 - TC-03, TC-04, and TC-05: Adjusted boundary IDs to match actual database seed limits ($N = 5$, $N - 1 = 4$, $N + 1 = 6$).
 - All error test cases (TC-05 to TC-30, TC-35): Removed hardcoded `{"error": "<string>"}` string assertions; updated assertions to check HTTP status codes and JSON response structure only.
 - TC-33: Added Pre-request Script to log in via `POST /api/login` with regular user credentials and set `Authorization: Bearer <token>` dynamically.
 - TC-34 & TC-36: Pointed target endpoint to `id=2` to validate schema completeness, strict numeric type enforcement (`typeof price === 'number'`), and positive domain bounds (`price > 0`).
 - TC-35: Replaced the NoSQL injection payload with an error schema validation test case for non-existent product IDs (`id=99999`).

Pool B:

- API endpoint: `POST /api/checkout`
- Feature: FR-08 - Checkout
- Audit: All test cases are VALID, except for TC-25 (INVALID), TC-26 (INVALID), TC-03 (INCOMPLETE), TC-21 (INCOMPLETE), and TC-35 (INCOMPLETE).

- TC-25 is INVALID because relying on a naturally time-expired token introduces a non-deterministic timing dependency that cannot run consistently across automated environments.
 - TC-26 is INVALID because modifying the JWT payload client-side breaks the cryptographic signature, making it redundant with TC-24 (Invalid/Malformed Token) rather than verifying role authorization.
 - TC-03 is INCOMPLETE because it assumes adding a synthetic product with `id=2` and price `1 VND` to the cart, which conflicts with actual existing product prices in the database and causes a cart total mismatch.
 - TC-21 is INCOMPLETE because the system does not provide an endpoint to clear or delete the cart state deterministically for an existing user.
 - TC-35 is INCOMPLETE because adding a synthetic product priced at `9007199254740991 VND` directly to `/api/cart` fails backend product existence validation.
- Correction:
 - TC-26: Replaced with testing Checkout under an Admin account (`admin@eshop.com`) to verify order placement behavior across roles.
 - TC-03: Update Pre-request Script to use an existing product with a single quantity or create a valid product via the Admin API before adding it to the cart.
 - TC-21: Update Pre-request Script to dynamically register and log in with a fresh unique user account (`POST /api/register`) to guarantee an empty cart state before checkout.
 - TC-35: Update Pre-request Script to use a valid high-value product from the database and increase its `quantity` to reach the safe integer boundary (`Number.MAX_SAFE_INTEGER`).

Pool C:

- API endpoint: `POST /api/admin/import-products`
- Feature: FR-16 - Import Products
- Audit: All test cases are VALID, except for TC-07 (INCOMPLETE), TC-23 (INCOMPLETE), TC-29 (INCOMPLETE), and TC-36 (INCOMPLETE).
 - TC-07 is INVALID because sending an object directly in Postman without setting the collection variable or explicitly handling serialization can cause pre-execution parsing mismatch.
 - TC-23 is INCOMPLETE because the payload specifies an abbreviated ellipsis string (`"aaaaaaaa...(5000 'a' characters total)"`) instead of generating a concrete 5000-character string via dynamic pre-request scripting (`"a".repeat(5000)`).
 - TC-29 is INCOMPLETE because using `garbage.invalid.token123` as a Bearer token tests signature decoding failure, but lacks the dual assertion for `401 Unauthorized` or `403 Forbidden` standard middleware responses.
 - TC-36 is INCOMPLETE because the test assertion strictly expects `Content-Type: application/json` on an unmapped HTTP GET route, whereas standard Express routing returns an HTML 404 page (`text/html`) when a route handler is absent.
- Correction:
 - TC-23: Update the Pre-request Script to dynamically construct the 5000-character string via `pm.collectionVariables.set("longProductName", "a".repeat(5000))` and reference

`{{longProductName}}` in the JSON body.

- TC-29: Update Expected HTTP Status to `401` or `403` to accurately accommodate standard JWT authentication middleware behavior across environments.
- TC-36: Update Expected Response Body assertion to verify that the GET method is rejected with `404 Not Found` or `405 Method Not Allowed` without enforcing `application/json` content-type on unrouted endpoints.

Test case extension

Pool A:

TC-37 — Objective: Retrieve soft-deleted / inactive product via public product detail endpoint

Pre-conditions: Product id=5 has `is_active=0` in database

HTTP Method & URL: `GET http://localhost:3000/api/products/5`

Request Headers: None (public endpoint)

Request Body: None (GET request)

Pre-request Script: None

Expected HTTP Status: 404 Not Found

Expected Response Body / Assertion: Inactive/hidden products are not returned; JSON format; does not expose soft-deleted item details

Covers: State / Data integrity — Soft-delete visibility constraint

TC-38 — Objective: Blind Time-Based SQL Injection via heavy SQLite computational payload in path parameter `:id`

Pre-conditions: None (system running with SQLite database backend)

HTTP Method & URL: `GET http://localhost:3000/api/products/1%20AND%201=LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(5000000))))`

Request Headers: None (public endpoint)

Request Body: None (GET request)

Pre-request Script: None

Expected HTTP Status: 400 Bad Request

Expected Response Body / Assertion: Response time < 500ms; no computational delay executed; no SQLite database error or table details exposed

Covers: Security testing — SEC-05 Blind Time-Based SQL Injection / Parameter sanitization

Pool B:

TC-35 — Objective: Direct product price injection in checkout request body (Price Tampering & Parameter Injection)

Pre-conditions: User registered; standard cart present (total amount = 200000)

HTTP Method & URL: **POST** `http://localhost:3000/api/checkout`

Request Headers: **Authorization:** `Bearer <valid_token>`, **Content-Type:** `application/json`

Request Body:

```
{
  "total_amount": 200000,
  "shipping_address": "123 Le Loi, TP.HCM",
  "price": 1000
}
```

Pre-request Script: Login `test@domain.com` / `Password123!` to obtain Bearer token; execute **POST** `/api/cart` to add items totaling 200,000 VND

Expected HTTP Status: 200 OK or 400 Bad Request

Expected Response Body / Assertion: **Content-Type:** `application/json`; the injected `price` parameter is ignored or rejected; order is created with the legitimate cart total (`total_amount == 200000`) without price tampering

Covers: Security / Data integrity — Price Tampering & Parameter Injection Prevention

TC-36 — Objective: Mass Assignment protection against injected administrative and order state flags

Pre-conditions: User registered; standard cart present (total amount = 200000)

HTTP Method & URL: **POST** `http://localhost:3000/api/checkout`

Request Headers: **Authorization:** `Bearer <valid_token>`, **Content-Type:** `application/json`

Request Body:

```
{
  "total_amount": 200000,
  "shipping_address": "123 Le Loi, TP.HCM",
  "status": "delivered",
  "is_admin": true,
  "is_paid": true
}
```

Pre-request Script: Login `test@domain.com` / `Password123!` to obtain Bearer token; execute `POST /api/cart` to add items totaling 200,000 VND

Expected HTTP Status: 200 OK or 400 Bad Request

Expected Response Body / Assertion: `Content-Type: application/json`; unauthorized administrative fields are ignored or rejected; created order retains default initial status (`status == "pending"`) and user privileges are not elevated

Covers: Robustness / Security — Mass Assignment & Privilege Escalation Prevention

Here is the complete description for **TC-07** written entirely in English:

Pool C:

TC-07 — Objective: Verify transaction rollback / atomicity when importing a batch containing a foreign key violation

Pre-conditions: Admin account exists; `category_id=1` exists; `category_id=99999` does not exist in the database

HTTP Method & URL: `POST http://localhost:3000/api/admin/import-products`

Request Headers: `Authorization: Bearer {{adminToken}}`, `Content-Type: application/json`

Request Body:

```
{
  "products": [
    {
      "name": "SP Atomicity Valid",
      "price": 150000,
      "description": "Valid product prior to error",
      "imageUrl": "http://example.com/valid.png",
      "category_id": 1
    },
    {
      "name": "SP Atomicity Invalid FK",
      "price": 200000,
      "description": "Invalid product with foreign key error",
      "imageUrl": "http://example.com/invalid.png",
      "category_id": 99999
    }
  ]
}
```

Pre-request Script:

```
const baseUrl = pm.collectionVariables.get("baseUrl") || "http://localhost:3000";

// Login as Admin to acquire bearer token
pm.sendRequest({
  url: baseUrl + '/api/login',
  method: 'POST',
  header: { 'Content-Type': 'application/json' },
  body: {
    mode: 'raw',
    raw: JSON.stringify({
      email: "admin@eshop.com",
      password: "Admin123!"
    })
  }
}, function (err, res) {
  if (!err && res.code === 200) {
    const token = res.json().token;
    pm.collectionVariables.set("adminToken", token);
    pm.request.headers.upsert({
      key: 'Authorization',
      value: 'Bearer ' + token
    });
  }
});
```

Expected HTTP Status: 400 Bad Request or 422 Unprocessable Entity

Expected Response Body / Assertion:

- **Content-Type:** application/json
- Status code is 400 or 422
- The entire transaction is aborted and rolled back; the valid product **SP Atomicity Valid** is not persisted in the database (verified via **GET /api/products?search=SP Atomicity Valid** returning an empty array)

Covers: Business Logic / Transaction Integrity — Database Atomicity & Rollback Verification on Batch Import

Why the AI missed these test cases

These test cases were extended to cover deeper data integrity, parameter tampering, and backend-specific edge cases. The AI missed them because it was constrained to a fixed 36-test-case budget while balancing standard equivalence partitioning, boundary values, and general security checks, leaving out complex combinations such as batch transaction atomicity, soft-delete constraints, and SQLite-specific blind SQL injection.

Test case execution:

- [GitHub repository link](#)

- [Excel report](#)
- [Bug report](#)
- List of Postman features used: workspace, collection, variable, pre-request scripts, postman cli(via CI).
- [CI/CD report](#)

Appendices

Appendix A

[AI Audit Report](#)

[AI Critique](#)

Appendix B

[Self-assessment & Test summary report](#)